Kingdom of Saudi Arabia
Ministry of education
Umm Al Qura University

جامعة أم القرى
UMM AL-QURA UNIVERSITY

المملكة العربية السعودية
وزارة التعليم
جامعة أم القرى

Group Project Assignment:
Data Engineering Pipeline

Members:

Samar Amer Asiri 444000717

Raghad Ibrahim Alsubhi 4440003965

Aya Adnan Babkoor 444002180

Rema Khalid Alghamdi 444001279

Group: 3

Table of Contents:

Data Description and Rationale for Choice:

Our project involves detailed sales data containing unique identifiers for orders and products, along with product categories, quantities sold, unit and total prices, order dates, customer information, payment methods, and order statuses.
This data combines both numerical and categorical values, enabling precise analysis of sales performance and customer behavior, which supports optimizing business operations and making data-driven decisions.
We chose this sales dataset because sales data is fundamental for any business, offering valuable insights into customer preferences, product trends, and revenue streams. Analyzing such data helps companies improve inventory management, marketing strategies, and overall operational efficiency.

Design Overview:

This section provides a high-level summary of the overall architecture and workflow of the project. It outlines the four main phases involved in processing and analyzing sales data: relational database design and normalization, NoSQL data transformation and querying, stream processing using PySpark, and integration with reporting. Each phase builds upon the previous one to create an efficient data pipeline that enables comprehensive data management and insightful analysis.

Phase-wise Implementation and Outputs:

Phase 1 - Task 1: Upload and Preview Sales Data CSV
Description:
In this step, we uploaded the sales data CSV file into the Google Colab environment. We then used the Pandas library to read the data, display the column names, and preview the first 5 rows to verify the data integrity.

لم يتم اختيار أي ملف اختيار الملفات Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving sales_5000.csv to sales_5000 (1).csv
```

	Order ID	Product ID	Product Category	Quantity	Unit Price	Total Price	Order Date	Customer ID	Payment Type	Order Status
0	873ff350-854d-4bd8-aebf-6cc5e1a6d3b7	26ab320e-62f2-4203-be68-f8740085d796	Sports & Outdoors	4	154.050509	616.202036	2023-08-02	292fe8cf-4d60-437b-8488-4c225e84d48d	Credit Card	Completed
1	7cb2641e-ea6b-4faf-a7ab-24992d5573fa	8c0d7ba9-1e6b-412a-8359-97d964ad19e5	Home & Kitchen	5	275.539908	1377.699538	2023-09-03	a026178d-e3ed-4d45-a49c-8336479b2114	PayPal	Refunded
2	94e0de8a-419d-46e1-9e7c-65c730b89c65	5d65b6c7-1894-4207-b7e3-b53cc2d5e5e3	Beauty & Health	3	56.410098	169.230294	2023-12-06	808e8010-1552-42dd-a9c5-eacd4fde3283	Debit Card	Cancelled
3	88cc8ccf-f018-46ac-ba3b-f36b5cf4e429	b3f6c6a5-d3cb-49a4-b53b-3996be815c28	Books	2	176.410906	352.821812	2023-07-22	6184896b-4fb2-4ae0-9851-65033dae1319	PayPal	Pending
4	eb520575-720e-456e-9dc5-c2c3bb887bfc	fed52dbf-49c6-4128-8b18-c36e50184a04	Electronics	3	409.892144	1229.676431	2023-04-08	24689017-68fc-42ba-8ae6-f0ad7155e0d6	Credit Card	Cancelled

Phase 1 - Task 2: Design and Create Relational Database Tables
Description:

In this step, we designed and created a normalized relational database schema for the sales data. Four main tables were created to represent distinct entities: Customers, Products, Orders, and OrderDetails.
We applied normalization principles up to the Third Normal Form (3NF) to organize data efficiently, reduce redundancy, and maintain data integrity. Relationships were established using primary and foreign keys.

```
DONE: 4 tables created (Customers, Products, Orders, OrderDetails)
```

Phase 1 - Task 3: Insert Data into Tables

Description:

In this task, we populated the relational database tables with unique sales data. The insertion process ensured no duplicates and maintained proper relationships between Customers, Products, Orders, and OrderDetails tables to preserve data integrity.

Customers Table (First 5 rows):

	CustomerID
0	292fe8cf-4d60-437b-8488-4c225e84d48d
1	a026178d-e3ed-4d45-a49c-8336479b2114
2	808e8010-1552-42dd-a9c5-eacd4fde3283
3	6184896b-4fb2-4ae0-9851-65033dae1319
4	24689017-68fc-42ba-8ae6-f0ad7155e0d6

Products Table (First 5 rows):

	ProductID	ProductCategory	UnitPrice
0	26ab320e-62f2-4203-be68-f8740085d796	Sports & Outdoors	154.050509
1	8c0d7ba9-1e6b-412a-8359-97d964ad19e5	Home & Kitchen	275.539908
2	5d65b6c7-1894-4207-b7e3-b53cc2d5e5e3	Beauty & Health	56.410098
3	b3f6c6a5-d3cb-49a4-b53b-3996be815c28	Books	176.410906
4	fed52dbf-49c6-4128-8b18-c36e50184a04	Electronics	409.892144

Products Table (First 5 rows):

	ProductID	ProductCategory	UnitPrice
0	26ab320e-62f2-4203-be68-f8740085d796	Sports & Outdoors	154.050509
1	8c0d7ba9-1e6b-412a-8359-97d964ad19e5	Home & Kitchen	275.539908
2	5d65b6c7-1894-4207-b7e3-b53cc2d5e5e3	Beauty & Health	56.410098
3	b3f6c6a5-d3cb-49a4-b53b-3996be815c28	Books	176.410906
4	fed52dbf-49c6-4128-8b18-c36e50184a04	Electronics	409.892144

Orders Table (First 5 rows):

	OrderID	OrderDate	CustomerID	PaymentType	OrderStatus
0	873ff350-854d-4bd8-aebf-6cc5e1a6d3b7	2023-08-02	292fe8cf-4d60-437b-8488-4c225e84d48d	Credit Card	Completed
1	7cb2641e-ea6b-4faf-a7ab-24992d5573fa	2023-08-03	a026178d-e3ed-4d45-a49c-8336479b2114	PayPal	Refunded
2	94e0de8a-419d-46e1-9e7c-65c730b89c65	2023-12-06	808e8010-1552-42dd-a9c5-eacd4fde3283	Debit Card	Cancelled
3	88cc8ccf-f018-46ac-ba3b-f36b5cf4e429	2023-07-22	6184896b-4fb2-4ae0-9851-65033dae1319	PayPal	Pending
4	eb520575-720e-456e-9dc5-c2c3bb887bfc	2023-04-08	24689017-68fc-42ba-8ae6-f0ad7155e0d6	Credit Card	Cancelled

OrderDetails Table (First 5 rows):

	OrderDetailID	OrderID	ProductID	Quantity	UnitPrice	TotalPrice
0	1	873ff350-854d-4bd8-aebf-6cc5e1a6d3b7	26ab320e-62f2-4203-be68-f8740085d796	4	None	616.202036
1	2	7cb2641e-ea6b-4faf-a7ab-24992d5573fa	8c0d7ba9-1e6b-412a-8359-97d964ad19e5	5	None	1377.699538
2	3	94e0de8a-419d-46e1-9e7c-65c730b89c65	5d65b6c7-1894-4207-b7e3-b53cc2d5e5e3	3	None	169.230294
3	4	88cc8ccf-f018-46ac-ba3b-f36b5cf4e429	b3f6c6a5-d3cb-49a4-b53b-3996be815c28	2	None	352.821812
4	5	eb520575-720e-456e-9dc5-c2c3bb887bfc	fed52dbf-49c6-4128-8b18-c36e50184a04	3	None	1229.676431

Phase 1 - Task 4: CRUD Operations

Description:

This task covers fundamental CRUD operations on the sales database, including reading completed orders, inserting multiple real orders, updating order statuses, and deleting specific orders. These steps demonstrate effective data management and integrity in the system.

READ: First 10 Completed Orders

	OrderID	CustomerID	OrderDate	PaymentType	OrderStatus
0	873ff350-854d-4bd8-aebf-6cc5e1a6d3b7	292fe8cf-4d60-437b-8488-4c225e84d48d	2023-08-02	Credit Card	Completed
1	b80f6f1d-8052-44a1-9e30-35fd58f6a77a	c1fd1416-d468-4cb2-8edb-da436fc8caa4	2023-05-17	Credit Card	Completed
2	542d294e-4363-4dcb-99d4-5caa30e5c9c3	f3ac55a5-33e7-4cd1-985f-d312d4f303bf	2023-05-24	PayPal	Completed
3	7493ca62-ef6b-42b4-ae50-59f279c1e0a2	2ce2271d-423f-4fc7-9b22-c433e90acdf0	2023-06-11	Credit Card	Completed
4	a35b2d5f-9b7d-46be-9cd0-d65752ad8c8b	16b3a153-a3d6-4531-b771-01e01fbcdbf0	2023-04-22	Debit Card	Completed
5	187c575d-20de-4c47-8b73-dd0261190b92	6c8314e0-9415-41e1-b609-1abafaeadbf2	2023-01-26	PayPal	Completed
6	3ec631ce-fa82-4f22-ad99-43ec72dee857	a19e5b44-3753-42fd-9ef4-8c25e90dcc05	2023-09-05	Credit Card	Completed
7	7b16ddcb-e572-4cda-8454-3bd4df02da49	e75e4f8e-cc1c-4ce4-a77d-8e3ea05facc6	2023-01-19	Credit Card	Completed
8	672ae50a-dade-4ea4-8e87-333ffe6e6188	0a1383bf-a95c-49f9-87c2-b5423c7e0c01	2023-12-25	Credit Card	Completed
9	9789ef75-7c78-4244-a8c1-26482f5a9263	8ffab66b-51d9-47a2-8880-d3a213e3f02a	2023-09-12	PayPal	Completed

INSERT: Multiple orders and details added
Orders Sample:

	OrderID	OrderDate	CustomerID	PaymentType	OrderStatus
0	873ff350-854d-4bd8-aebf-6cc5e1a6d3b7	2023-08-02	292fe8cf-4d60-437b-8488-4c225e84d48d	Credit Card	Completed
1	7cb2641e-ea6b-4faf-a7ab-24992d5573fa	2023-09-03	a026178d-e3ed-4d45-a49c-8336479b2114	PayPal	Refunded
2	94e0de8a-419d-46e1-9e7c-65c730b89c65	2023-12-06	808e8010-1552-42dd-a9c5-eacd4fde3283	Debit Card	Cancelled
3	88cc8ccf-f018-46ac-ba3b-f36b5cf4e429	2023-07-22	6184896b-4fb2-4ae0-9851-65033dae1319	PayPal	Pending
4	eb520575-720e-456e-9dc5-c2c3bb887bfc	2023-04-08	24689017-68fc-42ba-8ae6-f0ad7155e0d6	Credit Card	Cancelled
5	9208a712-e102-4f97-a0e0-345ffc6744cb	2023-07-03	f172dd53-ebd6-49cd-8545-3853788e142b	PayPal	Cancelled
6	ada3eefe-8782-4af1-b248-e4aa81a32cf0	2023-03-10	eeafe5a1-d9be-4edc-9881-8b60f5eeca86d	Credit Card	Pending
7	56866a60-0db6-4e10-9d19-9fbe6ecec562	2023-12-20	8f06c738-60f9-46ff-9279-25d12da8f9b7	Credit Card	Refunded
8	1e0c36b0-1c2b-4ad5-a479-b0ea24fb4824	2023-10-21	d4ca907b-f84a-460c-9a4d-7819b166dffc	PayPal	Refunded
9	83747baf-e907-4db6-952d-2d8f9fa8f801	2023-02-19	fec4616d-5ebc-4351-8509-430bdf9c37c4	PayPal	Refunded

UPDATE: Status changed to 'Completed' for orders: ['ORDER1001', 'ORDER1002', 'ORDER1003']

	OrderID	OrderStatus
0	ORDER1001	Completed
1	ORDER1002	Completed
2	ORDER1003	Completed

DELETE: Order 'ORDER1001' and its details removed
Deleted Order Check (should be empty):

OrderID	OrderDate	CustomerID	PaymentType	OrderStatus

All CRUD operations completed successfully.

Phase 1 - Task 5: Apply Indexing and Perform Comprehensive JOIN Query

Description:

In this task, we measured and compared the execution time of a comprehensive JOIN query on sales data before and after applying database indexes on key columns. The indexes significantly improve the performance of JOIN operations across the Orders, Customers, OrderDetails, and Products tables. After running the optimized query, we displayed a sample of the joined data and saved the final dataset as a CSV file for further analysis.

```
Query time without indexes: 0.0665 seconds

Indexes created successfully.

Query time with indexes: 0.0513 seconds

Joined Data Sample (first 5 rows):
                                OrderID   OrderDate  \
0  873ff350-854d-4bd8-aebf-6cc5e1a6d3b7  2023-08-02
1  7cb2641e-ea6b-4faf-a7ab-24992d5573fa  2023-09-03
2  94e0de8a-419d-46e1-9e7c-65c730b89c65  2023-12-06
3  88cc8ccf-f018-46ac-ba3b-f36b5cf4e429  2023-07-22
4  eb520575-720e-456e-9dc5-c2c3bb887bfc  2023-04-08

                             CustomerID                             ProductID  \
0  292fe8cf-4d60-437b-8488-4c225e84d48d  26ab320e-62f2-4203-be68-f8740085d796
1  a026178d-e3ed-4d45-a49c-8336479b2114  8c0d7ba9-1e6b-412a-8359-97d964ad19e5
2  808e8010-1552-42dd-a9c5-eacd4fde3283  5d65b6c7-1894-4207-b7e3-b53cc2d5e5e3
3  6184896b-4fb2-4ae0-9851-65033dae1319  b3f6c6a5-d3cb-49a4-b53b-3996be815c28
4  24689017-68fc-42ba-8ae6-f0ad7155e0d6  fed52dbf-49c6-4128-8b18-c36e50184a04

     ProductCategory  Quantity   TotalPrice
0  Sports & Outdoors         4   616.202036
1     Home & Kitchen         5  1377.699538
2    Beauty & Health         3   169.230294
3              Books         2   352.821812
4        Electronics         3  1229.676431

Joined data saved as CSV to: data/sales_joined_output.csv
```

Phase 2 - Task 1: Transform part of the data into a document or key-value format

Description:

In this task, we loaded the joined sales data CSV file and selected relevant columns to transform the data into a document-like structure. We converted the selected tabular data into a list of dictionaries, preparing it for insertion into a NoSQL database.

```
[{'CustomerID': '292fe8cf-4d60-437b-8488-4c225e84d48d', 'ProductID': '26ab320e-62f2-4203-be68-f8740085d796', 'OrderID': '873ff350-854d-4bd8-aebf
```

Phase 2 - Task 2: Insert data using a Python-based NoSQL library (e.g., pymongo , TinyDB)

Description:

This task involved creating a TinyDB NoSQL database and inserting the prepared document records into a table named "SalesRecords". We used TinyDB with caching to optimize performance and verified the insertion by printing sample records.

```
Requirement already satisfied: tinydb in /usr/local/lib/python3.11/dist-packages (4.8.2)
 SalesRecords Table (First 5 Rows):
{'CustomerID': '292fe8cf-4d60-437b-8488-4c225e84d48d', 'ProductID': '26ab320e-62f2-4203-be68-f8740085d796', 'OrderID': '873ff350-854d-4bd8-aebf
{'CustomerID': 'a026178d-e3ed-4d45-a49c-8336479b2114', 'ProductID': '8c0d7ba9-1e6b-412a-8359-97d964ad19e5', 'OrderID': '7cb2641e-ea6b-4faf-a7ab
{'CustomerID': '808e8010-1552-42dd-a9c5-eacd4fde3283', 'ProductID': '5d65b6c7-1894-4207-b7e3-b53cc2d5e5e3', 'OrderID': '94e0de8a-419d-46e1-9e7c
{'CustomerID': '6184896b-4fb2-4ae0-9851-65033dae1319', 'ProductID': 'b3f6c6a5-d3cb-49a4-b53b-3996be815c28', 'OrderID': '88cc8ccf-f018-46ac-ba3b
{'CustomerID': '24689017-68fc-42ba-8ae6-f0ad7155e0d6', 'ProductID': 'fed52dbf-49c6-4128-8b18-c36e50184a04', 'OrderID': 'eb520575-720e-456e-9dc5
```

Phase 2 - Task 3: Perform Basic Queries and Export Data from TinyDB

Description:

In this task, we executed basic queries on the TinyDB NoSQL database to retrieve records based on conditions such as Quantity greater than or equal to 4, TotalPrice less than 200, and records for a specific CustomerID. After querying, we exported all records from the database into an external JSON file for use in subsequent phases or backup purposes.

```
Records with Quantity >= 4:
{'CustomerID': '292fe8cf-4d60-437b-8488-4c225e84d48d', 'ProductID': '26ab320e-62f2-4203-be68-f8740085d796', 'OrderID': '873ff350-854d-4bd8
{'CustomerID': 'a026178d-e3ed-4d45-a49c-8336479b2114', 'ProductID': '8c0d7ba9-1e6b-412a-8359-97d964ad19e5', 'OrderID': '7cb2641e-ea6b-4faf
{'CustomerID': 'd4ca907b-f84a-460c-9a4d-7819b166dffc', 'ProductID': 'bf8cc408-2a84-476f-bba8-900c9da435eb', 'OrderID': '1e0c36b0-1c2b-4ad5
{'CustomerID': 'fec4616d-5ebc-4351-8509-430bdf9c37c4', 'ProductID': '0af4a42d-7c59-4d37-8c7a-c22392574b73', 'OrderID': '83747baf-e907-4db6
{'CustomerID': 'f3ac55a5-33e7-4cd1-985f-d312d4f303bf', 'ProductID': '33a72e34-6a7e-4003-b8f4-0dc18082d193', 'OrderID': '542d294e-4363-4dcb

Records with TotalPrice < 200:
{'CustomerID': '808e8010-1552-42dd-a9c5-eacd4fde3283', 'ProductID': '5d65b6c7-1894-4207-b7e3-b53cc2d5e5e3', 'OrderID': '94e0de8a-419d-46e1
{'CustomerID': 'c1fd1416-d468-4cb2-8edb-da436fc8caa4', 'ProductID': 'a9a3910b-76da-44a7-a7a8-c68d8088bcdd', 'OrderID': 'b80f6f1d-8052-44a1
{'CustomerID': '39617a0f-cace-41d7-86ac-96a2a64cd528', 'ProductID': '1935ec82-0dbc-4056-96ce-3e9aa610b66c', 'OrderID': 'b2d24e95-17a3-4d0b
{'CustomerID': '6414d972-776c-4000-aac9-d29eed1b5602', 'ProductID': '91f34197-85e8-4176-a0ac-76ece8c6cee2', 'OrderID': '10ac85f0-7dfb-427c
{'CustomerID': '15432b94-de22-4dcb-879e-6c63d476a085', 'ProductID': '49a91543-2207-49a9-9e98-360d1892557a', 'OrderID': '2d4f214e-02ea-48d0

Records for CustomerID = 292fe8cf-4d60-437b-8488-4c225e84d48d:
{'CustomerID': '292fe8cf-4d60-437b-8488-4c225e84d48d', 'ProductID': '26ab320e-62f2-4203-be68-f8740085d796', 'OrderID': '873ff350-854d-4bd8
{'CustomerID': '292fe8cf-4d60-437b-8488-4c225e84d48d', 'ProductID': '26ab320e-62f2-4203-be68-f8740085d796', 'OrderID': '873ff350-854d-4bd8
Data successfully exported to sales_records_export.json
```

Phase 3 - Task 1: Load JSON Data into Spark DataFrame

Description:

In this task, we initialized a Spark session and loaded the multi-line JSON file sales_records_export.json generated from Phase 2 into a Spark DataFrame. Using the multiLine=True option ensured proper parsing of the JSON file. We then displayed the first 5 rows to verify successful data loading and preview the sales records.

```
+--------------------+--------------------+--------------------+--------+------------------+
|          CustomerID|             OrderID|           ProductID|Quantity|        TotalPrice|
+--------------------+--------------------+--------------------+--------+------------------+
|292fe8cf-4d60-437...|873ff350-854d-4bd...|26ab320e-62f2-420...|       4| 616.2020360083505|
|a026178d-e3ed-4d4...|7cb2641e-ea6b-4fa...|8c0d7ba9-1e6b-412...|       5| 1377.699538425277|
|808e8010-1552-42d...|94e0de8a-419d-46e...|5d65b6c7-1894-420...|       3| 169.2302943628258|
|6184896b-4fb2-4ae...|88cc8ccf-f018-46a...|b3f6c6a5-d3cb-49a...|       2| 352.8218115631303|
|24689017-68fc-42b...|eb520575-720e-456...|fed52dbf-49c6-412...|       3|1229.6764310869437|
+--------------------+--------------------+--------------------+--------+------------------+
only showing top 5 rows
```

Phase 3 Task 2: Use PySpark to filter and process the data

Description:

In this task, we performed various data filtering and selection operations on the sales dataset loaded in Spark:

Filtered orders where the quantity purchased is greater than 3, highlighting large orders.

Filtered orders with a total price exceeding 1000 to identify high-value transactions.

Selected key columns — OrderID, CustomerID, ProductID, Quantity, and TotalPrice — to focus on important attributes.

Filtered orders for a specific customer by their CustomerID as an example of targeted data retrieval.

```
Orders with Quantity > 3:
+--------------------+--------------------+--------------------+--------+------------------+
|          CustomerID|             OrderID|           ProductID|Quantity|        TotalPrice|
+--------------------+--------------------+--------------------+--------+------------------+
|292fe8cf-4d60-437...|873ff350-854d-4bd...|26ab320e-62f2-420...|       4| 616.2020360083505|
|a026178d-e3ed-4d4...|7cb2641e-ea6b-4fa...|8c0d7ba9-1e6b-412...|       5| 1377.699538425277|
|d4ca907b-f84a-460...|1e0c36b0-1c2b-4ad...|bf8cc408-2a84-476...|       5|1315.1968572619671|
|fec4616d-5ebc-435...|83747baf-e907-4db...|0af4a42d-7c59-4d3...|       4| 464.2897717196084|
|f3ac55a5-33e7-4cd...|542d294e-4363-4dc...|33a72e34-6a7e-400...|       4| 1261.957045183035|
+--------------------+--------------------+--------------------+--------+------------------+
only showing top 5 rows

Orders with TotalPrice > 1000:
+--------------------+--------------------+--------------------+--------+------------------+
|          CustomerID|             OrderID|           ProductID|Quantity|        TotalPrice|
+--------------------+--------------------+--------------------+--------+------------------+
|a026178d-e3ed-4d4...|7cb2641e-ea6b-4fa...|8c0d7ba9-1e6b-412...|       5| 1377.699538425277|
|24689017-68fc-42b...|eb520575-720e-456...|fed52dbf-49c6-412...|       3|1229.6764310869437|
|d4ca907b-f84a-460...|1e0c36b0-1c2b-4ad...|bf8cc408-2a84-476...|       5|1315.1968572619671|
|27c25d39-7088-46a...|32fa48f8-32d5-42b...|20ddbbf2-8904-47c...|       3|1333.9888705665771|
|f3ac55a5-33e7-4cd...|542d294e-4363-4dc...|33a72e34-6a7e-400...|       4| 1261.957045183035|
+--------------------+--------------------+--------------------+--------+------------------+
only showing top 5 rows
```

```
Important Columns:
+--------------------+--------------------+--------------------+--------+------------------+
|             OrderID|          CustomerID|           ProductID|Quantity|        TotalPrice|
+--------------------+--------------------+--------------------+--------+------------------+
|873ff350-854d-4bd...|292fe8cf-4d60-437...|26ab320e-62f2-420...|       4| 616.2020360083505|
|7cb2641e-ea6b-4fa...|a026178d-e3ed-4d4...|8c0d7ba9-1e6b-412...|       5| 1377.699538425277|
|94e0de8a-419d-46e...|808e8010-1552-42d...|5d65b6c7-1894-420...|       3| 169.2302943628258|
|88cc8ccf-f018-46a...|6184896b-4fb2-4ae...|b3f6c6a5-d3cb-49a...|       2| 352.8218115631303|
|eb520575-720e-456...|24689017-68fc-42b...|fed52dbf-49c6-412...|       3|1229.6764310869437|
+--------------------+--------------------+--------------------+--------+------------------+
only showing top 5 rows

Orders for specific CustomerID:
+--------------------+--------------------+--------------------+--------+-----------------+
|          CustomerID|             OrderID|           ProductID|Quantity|       TotalPrice|
+--------------------+--------------------+--------------------+--------+-----------------+
|292fe8cf-4d60-437...|873ff350-854d-4bd...|26ab320e-62f2-420...|       4|616.2020360083505|
|292fe8cf-4d60-437...|873ff350-854d-4bd...|26ab320e-62f2-420...|       4|616.2020360083505|
+--------------------+--------------------+--------------------+--------+-----------------+
```

Phase 3 - Task 3: Save Filtered Results as CSV

Description:

This task involved saving the filtered dataset of large quantity orders into a CSV file using Spark's write API. The data was saved with overwrite mode to ensure any previous outputs are replaced. This CSV file can then be used for further processing or reporting.

```
Files saved to: ['part-00000-28095078-297e-442b-93fe-8961f98b61f7-c000.csv', '_SUCCESS', '.part-00000-28095078-297e-442b-93fe-8961f98b61f7-
```

Phase 4: Integration

In this final phase, all outputs from the previous phases were integrated into a comprehensive data pipeline. An interactive dashboard was designed to display the final data in an organized and user-friendly manner, facilitating real-time monitoring and analysis of sales performance.

The dashboard includes data from all three phases:

1- *providing a comprehensive view that combines the relational database records:*

Deploy

🛒 Sales Dashboard

📋 Phase 1: Sales (Relational DB - SQLite)

	CustomerID	OrderID	ProductID	Quantity	TotalPrice
0	292fe8cf-4d60-437b-8488-4c225e84d48d	873ff350-854d-4bd8-aebf-6cc5e1a6d3b7	26ab320e-62f2-4203-be68-f8740085d796	4	616.202
1	a026178d-e3ed-4d45-a49c-8336479b2114	7cb2641e-ea6b-4faf-a7ab-24992d5573fa	8c0d7ba9-1e6b-412a-8359-97d964ad19e5	5	1377.6995
2	d4ca907b-f84a-460c-9a4d-7819b166dffc	1e0c36b0-1c2b-4ad5-a479-b0ea24fb4824	bf8cc408-2a84-476f-bba8-900c9da435eb	5	1315.1969
3	fec4616d-5ebc-4351-8509-430bdf9c37c4	83747baf-e907-4db6-952d-2d8f8fa8f801	0af4a42d-7c59-4d37-8c7a-c22392574b73	4	464.2898
4	f3ac55a5-33e7-4cd1-985f-d312d4f303bf	542d294e-4363-4dcb-99d4-5caa30e5c9c3	33a72e34-6a7e-4003-b8f4-0dc18082d193	4	1261.957
5	49a6c4e4-b7f8-48b8-aa07-e88f93952e6f	b859120e-64e9-425e-99f3-d9bec6b99fb8	c1fe9334-83aa-419a-a338-84cf8650f4ca	4	329.346
6	63108c5d-0abb-42db-834f-42e66c2f2e1b	5baa9049-d059-4356-a0b3-642e55c84678	d657c80d-1c6f-4076-94ce-623baf755375	5	901.0016
7	3a8fe7ff-3683-4fc8-b77d-09a70714901c	c0f2ef36-114d-4862-815c-c8c63f10ba5c	dc6db32b-68ef-4885-bbb1-e5dac27f618a	5	1373.19
8	a19e5b44-3753-42fd-9ef4-8c25e90dcc05	3ec631ce-fa82-4f22-ad99-43ec72dee857	ec6cf0da-dfd1-4f3e-ad64-ff74d3d2e057	4	1149.8589
9	f5ec0e9f-c2af-4be8-9dd9-db9fb68babf8	6e94576a-3994-413c-b829-0fb7ce092ea5	ba058077-36d0-486f-bd25-fd8ce27bc8c8	4	434.0613

2- *NoSQL document data:*

🗂️ Phase 2: Products (NoSQL - TinyDB)

	order_id	product_id	quantity	total_price
0	0007cb07-b2af-4a37-aa43-fd650f6add56	37d58910-2ada-40a2-b378-fd00596cb528	5	157.1074
1	0021b9c8-e8f2-4820-a2bf-e62cc4535a0e	670d89cf-7467-4b1e-b0a8-eccb0f7b4eae	5	1251.2545
2	0045b060-ffa5-4bee-acd0-0e3db581ac49	0e60e0ae-7468-462f-8b60-5b5323e12e18	5	1266.4063
3	004d741a-e6f9-4dd0-b891-c38ac6719222	68aea003-a556-420b-b7c0-6fdfb4b05339	5	449.8463
4	00761d25-b27a-4902-966a-006646de428f	204b6d65-7273-4a0b-8e89-ece319c9a923	4	1880.9992
5	008d10c8-495d-4d26-bc86-7176af2641c4	35cd0738-c4e2-4d48-8ebb-306d065a63c1	4	1368.9852
6	00ece964-15e8-4820-9d6d-b1cb491e0dbb	90ee13c3-9f64-4038-932a-650f0afde1ff	4	1672.7447
7	01027835-c09f-4f79-8b26-3fa609197f04	5c5cadc6-ac7c-4f70-82c7-e18306b794f1	4	622.669
8	0111c21e-0c2c-4138-9a42-7cf9ae3776e4	af470d66-55a6-4072-ba5f-e8fc82888383	4	1844.7295
9	011fc2a2-4cc1-4e15-9d83-73f9b35a8fd5	3c27508e-5d94-4495-a922-6ef558314244	5	1116.4586

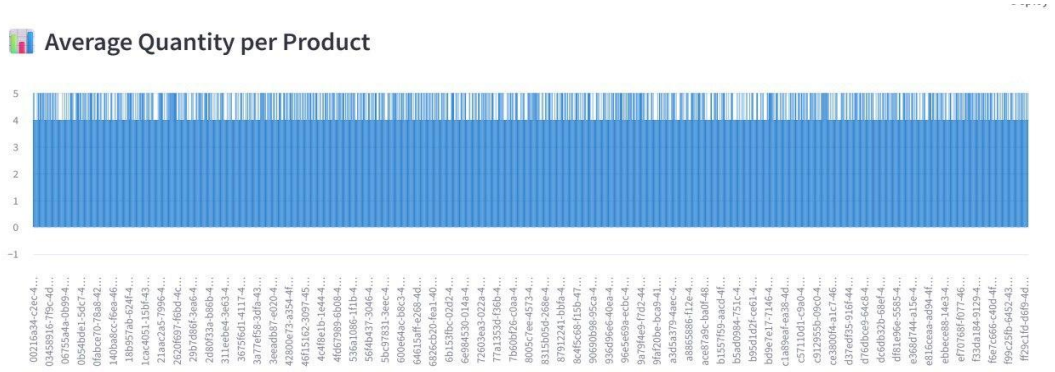
3- *stream-processed results*:

Phase 3: Streamed Sales (JSON Files)

	CustomerID	OrderID	ProductID	Quantity	TotalPrice
0	292fe8cf-4d60-437b-8488-4c225e84d48d	873ff350-854d-4bd8-aebf-6cc5e1a6d3b7	26ab320e-62f2-4203-be68-f8740085d796	4	616.202
1	a026178d-e3ed-4d45-a49c-8336479b2114	7cb2641e-ea6b-4faf-a7ab-24992d5573fa	8c0d7ba9-1e6b-412a-8359-97d964ad19e5	5	1377.6995
2	e75e4f8e-cc1c-4ce4-a77d-8e3ea05facc6	7b16ddcb-e572-4cda-8454-3bd4df02da49	2f0ad09f-cc53-4798-b853-b06f2a36f59d	4	1258.3926
3	e3e2c10c-435d-4c4f-bbd4-585007044897	38b114ad-0b71-4d6c-8517-d8f090f547eb	5cbde857-8414-4c6b-bc92-d4cc68f3fc73	4	1445.0971
4	52994298-4bd4-42e6-b71d-52db7c5a3bce	140c8374-fce6-4dda-9451-c54a29344329	098155f8-6fd3-400f-8b53-1c773cef6763	4	884.5451
5	59f16f85-655e-4f63-94bc-d085f88b63eb	9b340edd-a797-41c3-af49-4c2611439a7c	6b6805f5-dcb8-4ecd-93d1-6f9b54888d82	4	1953.9644
6	fa1c9f11-a85c-4363-ae55-9b21a764e23b	4a112eb1-e274-444f-8256-480666c7c995	3bf921e1-42c8-4847-a49b-c1fc9b55195e	5	1506.3851
7	63dbb6fb-bcdc-420f-8676-0475bd7e9f72	deef01cc-f60a-4ba1-8936-31c991c12002	68fe639e-9526-4be7-ad62-6b228c6717a6	5	1512.6884
8	b6d1a36d-8e78-4144-89c6-9518ef9aa435	48bab174-75e4-4e97-a817-27dba7476afc	32a90b30-8639-4edb-b376-a20ecbf130e4	4	450.3708
9	2f4c1d28-2322-47aa-9c14-049507e8a703	70499a84-ff4b-42cf-a03e-54789ca87ee6	a1ea9058-6142-446d-9809-9fe8a4e2700f	5	1716.2511

The dashboard includes visualizations representing data from all three phases:

1- *Average Quantity per Product: This chart shows the average quantity sold for each product, helping identify popular items.*



2- *Average Price per Product: This chart displays the average unit price of each product, useful for understanding pricing trends across product categories.*

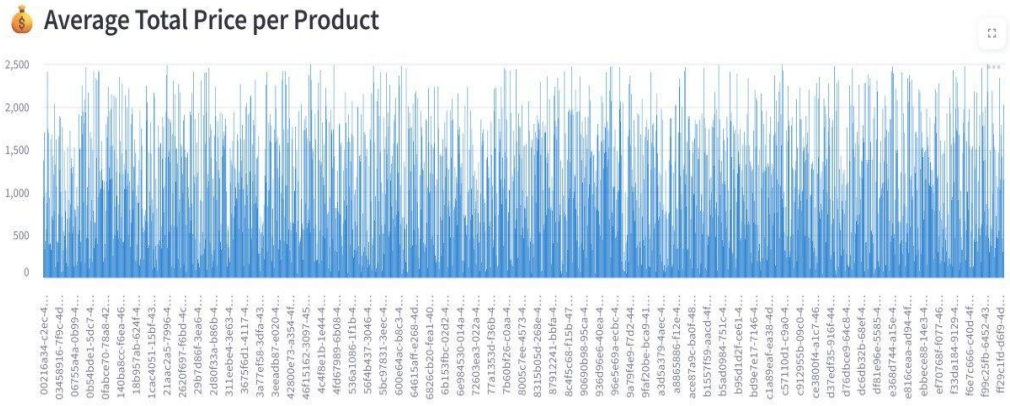


Diagram:

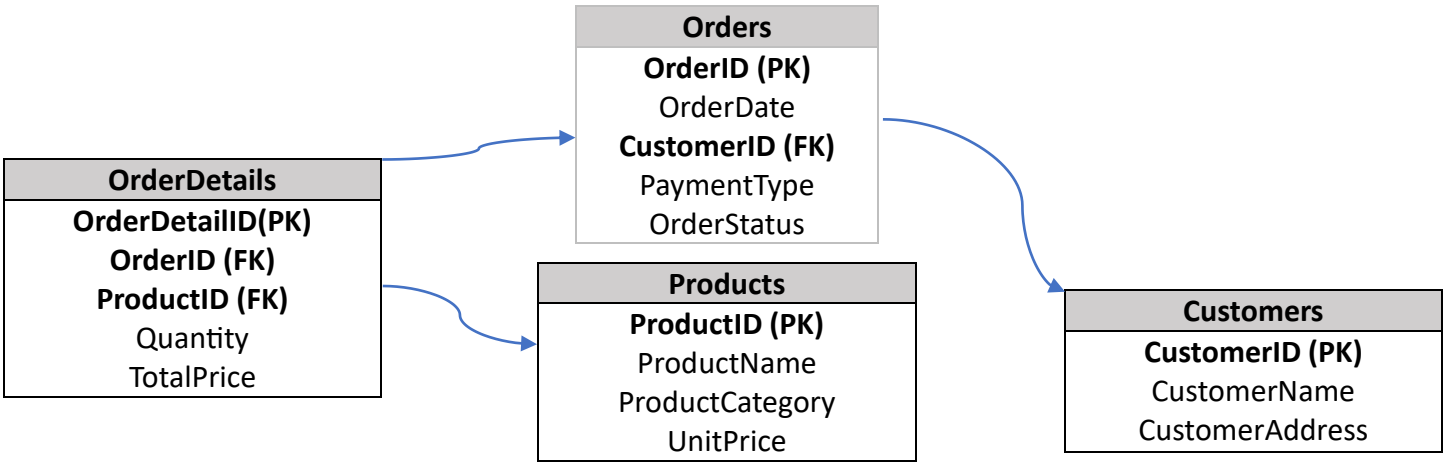
Normalization:

Normalization organizes database tables to reduce data duplication and improve integrity. It splits one big table into smaller related ones.

Before Normalization:
All sales data was in one big table causing redundancy and inconsistency.

SalesData	
PK	OrderID
	OrderDate
	CustomerID
	CustomerName
	CustomerAddress
	ProductID
	ProductName
	ProductCategory
	UnitPrice
	Quantity
	TotalPrice
	PaymentType
	| OrderStatus

After Normalization (3NF):
Data is split into related tables: Orders, Customers, Products, OrderDetails. Keys link tables to remove duplicates and maintain integrity.



Lessons Learned:

1. The importance of designing a well-structured database: We learned how applying normalization techniques improves storage efficiency and speeds up queries in relational databases.
2. Data integration from different sources: The experience of handling data from both relational and NoSQL sources taught us how to unify and organize data for more effective analysis.
3. Using stream processing techniques: We learned how to work with streaming data using PySpark, enabling fast and flexible real-time data analysis.

Challenges Faced:

1. Handling different data formats: We faced difficulties in converting data between different formats (CSV, JSON) and ensuring compatibility with each phase of the project.
2. Issues loading and processing data in PySpark: We encountered errors related to reading JSON files due to file formatting and schema mismatches, which required restructuring the data.
3. Synchronizing data saving and retrieval between phases: Managing the data flow across different phases was challenging, especially when saving results for proper reuse in subsequent stages.